A11 - Refatore sem mudar o comportamento

Iniciar tarefa

- Vencimento Domingo por 23:59
- Pontos 2
- Enviando uma caixa de entrada de texto
- Disponível 12 Out em 0:00 - 8 Nov em 23:59

Atividade prática • Tratamento da Dívida Técnica e Refatoração

Refatore sem mudar o comportamento

Nesta atividade, você vai decidir **quais dívidas merecem tratamento agora**, estabelecer uma baseline, refatorar um pequeno sistema em passos curtos e reunir evidências de que **a estrutura mudou sem alterar o comportamento observado**.

> 🎯 Objetivo
>
> Aplicar os conceitos de **tratamento proporcional da Dívida Técnica, refactoring, preservação de comportamento, pequenos passos e testes como rede de segurança**. O objetivo não é "deixar o código perfeito", mas executar uma intervenção delimitada e justificar por que ela faz sentido.

📦 Material da atividade

Um sistema pequeno com dívida registrada e testes prontos

Baixe o arquivo **Laboratorio_S12_Refatoracao.zip (https://pucminas.instructure.com/courses/293682/files/18805475?wrap=1)** ⬇ **(https://pucminas.instructure.com/courses/293682/files/18805475/download?download_frd=1)** , disponibilizado junto desta atividade, e extraia seu conteúdo em uma pasta no computador.

REQUISITO	SISTEMA	REDE DE SEGURANÇA
Python 3.10+	**Checkout de vendas**	**7 testes**
nenhum pacote externo	desconto, frete, imposto e mensagem	unittest + snapshots

O pacote inclui uma cópia do código original em `baseline/`. O comando `python lab.py diff` compara sua versão atual com esse baseline, e `python lab.py reset` restaura o arquivo original caso você precise recomeçar.

PARTE 1 • Antes de tocar no código

Nem toda dívida precisa receber a mesma resposta

Abra o arquivo `contexto.md`. Ele descreve três itens de Dívida Técnica:

Item	Contexto	Sua decisão
TDI-01	`checkout.py`: muda com frequência, nova regra prevista e testes existentes.	Eliminar? Reduzir? Mitigar? Aceitar? Postergar?
TDI-02	`legacy_export.py`: código ruim, mas quase congelado e com aposentadoria próxima.	Eliminar? Reduzir? Mitigar? Aceitar? Postergar?
TDI-03	Integração fiscal crítica presa a SDK externo ainda indisponível; já há monitoramento e testes de contrato.	Eliminar? Reduzir? Mitigar? Aceitar? Postergar?

Registre sua decisão antes de continuar.

Para cada TDI, indique a **resposta principal** e justifique em 2 ou 3 frases. Uma situação pode combinar ações, mas você deve deixar claro qual estratégia considera predominante.

PARTE 2 • Baseline

Antes de refatorar, descubra o que precisa continuar igual

No terminal, dentro da pasta do laboratório, execute:

```
python lab.py run
python -m unittest discover -s tests -v
python lab.py inspect
python lab.py snapshot before.json
```

Observe o comportamento

Quais são os resultados para clientes REGULAR, SILVER e GOLD? Em que condição o frete se torna gratuito?

Observe a estrutura

Quantas linhas, condicionais e laços o comando `inspect` encontra dentro de `checkout()`?

Importante: as medidas exibidas por `inspect` descrevem a estrutura da função. Elas **não são uma nota de qualidade** e não provam que existe ou deixou de existir Dívida Técnica.

PARTE 3 • Planeje a intervenção

Escolha pequenos passos — não uma reescrita

Abra `src/checkout.py`. A função concentra várias responsabilidades. Planeje **pelo menos três pequenas transformações** que tornem o código mais fácil de compreender e modificar, sem alterar a regra de negócio.

IDEIA POSSÍVEL

Extract Function

separar o cálculo do subtotal

IDEIA POSSÍVEL

Extract Function

separar desconto, frete ou imposto

IDEIA POSSÍVEL

Extract Function

isolar a formatação da mensagem

Você **não precisa seguir exatamente essas sugestões**. O importante é que as mudanças sejam pequenas, tenham intenção clara e preservem a interface pública de `checkout(order)`.

PARTE 4 • Refatore em pequenos passos

Depois de cada transformação, verifique

Faça **uma transformação de cada vez**. Depois de cada passo, execute:

```
python -m unittest discover -s tests -v
```



Se algum teste falhar...

Pare. Investigue a última transformação antes de continuar. O objetivo não é “consertar o teste” para fazê-lo aceitar um comportamento novo; nesta atividade, **o comportamento existente deve ser preservado**.

PARTE 5 • Compare antes e depois

A estrutura mudou. E o comportamento?

Quando terminar sua refatoração, execute:

```
python lab.py inspect
python lab.py snapshot after.json
python lab.py compare before.json after.json
python lab.py diff
```

Se os snapshots forem idênticos

Você possui evidência de que **os cenários observados** produziram as mesmas saídas antes e depois.

Isso não prova tudo

Os snapshots e testes cobrem os cenários fornecidos. Comportamentos não exercitados podem continuar fora da evidência.

PARTE 6 • O critério de parada

Agora olhe para `legacy_export.py`. Você deve refatorá-lo também?

Abra rapidamente `src/legacy_export.py`. Ele também contém código pouco elegante. Mas releia o contexto do TDI-02 antes de decidir tocar nele.

Relevância: refatorá-lo ajuda a tarefa que você está executando agora?

Horizonte: o componente continuará existindo tempo suficiente para recuperar o investimento?

Segurança: há evidência suficiente para uma intervenção estrutural sem mudar comportamento?

Retorno: o benefício esperado justifica ampliar o escopo do laboratório para esse arquivo?

A atividade não pede que você refatore `legacy_export.py`.

A intenção é justamente exercitar o critério de parada: **código imperfeito não cria, por si só, obrigação de intervenção imediata**.

PARTE 7 • A análise

O que seu experimento demonstrou?

Responda às perguntas abaixo com base no que você executou:

1. Qual estratégia de tratamento você escolheu para cada um dos três TDIs? Por quê?

2. Que comportamento você considerou como baseline antes de refatorar `checkout()`?

3. Quais refactorings você realizou? Descreva pelo menos três passos e a intenção de cada um.

4. Que evidências indicam que o comportamento foi preservado? Quais limites essas evidências possuem?

5. O que mudou no resultado de `python lab.py inspect`? Por que essa mudança estrutural, sozinha, não prova que o código “ficou melhor”?

6. Em que sentido você considera que o TDI-01 foi eliminado, reduzido ou apenas reorganizado?

7. Por que continuar refatorando `legacy_export.py` poderia ser um exemplo de ultrapassar o critério de parada?

📤 Entrega

Relatório curto + evidências da intervenção

No campo de resposta da atividade, entregue uma análise de aproximadamente **600 a 800 palavras**, organizada pelas sete perguntas acima.

Ao final, copie e cole também:

- a saída final de `python lab.py inspect`;
- a saída de `python lab.py compare before.json after.json`;
- a saída de `python lab.py diff`.

Não é necessário anexar o projeto inteiro. O `diff` mostra as transformações realizadas e permite discutir a intervenção sem depender de upload adicional de código.

Guarde esta ideia

Refactoring não é "mexer até o código parecer melhor". É uma intervenção estrutural deliberada, feita em passos pequenos, com evidência de preservação do comportamento e com um objetivo suficientemente claro para sabermos quando parar.

Referências técnicas para o laboratório:

FOWLER, Martin. *Refactoring: Improving the Design of Existing Code*. 2. ed. Addison-Wesley, 2018. **Página oficial → (https://martinfowler.com/books/refactoring.html)**

FOWLER, Martin. *Catalog of Refactorings*. Refactoring.com. **Catálogo → (https://refactoring.com/catalog/)**

PYTHON SOFTWARE FOUNDATION. *unittest — Unit testing framework*. **Documentação → (https://docs.python.org/3/library/unittest.html)**

PYTHON SOFTWARE FOUNDATION. *difflib — Helpers for computing deltas*. **Documentação → (https://docs.python.org/3/library/difflib.html)**

PUC Minas • Engenharia de Software • Dívida Técnica e Evolução de Software